DevOps Explanation Request

Alright vicky 😈
DAY-29: Kubernetes Storage — this is where people *think* they understand… until data disappears and panic begins.

I'll follow **the same flow** you like:

👉 **way → problem → how Kubernetes solves it → hands-on → edge cases → traps**

☠️ Way (Container-only storage – the trap)

Originally, containers stored data **inside the container filesystem**.

What happens then?

- Container restarts × → data gone
- Pod deleted × → data gone
- Node crash × → data gone

Containers are **ephemeral by design**.
Data is **not**.

So Kubernetes had to separate:

> **Application lifecycle ≠ Data lifecycle**

💣 Problem

1. Apps like **Redis, MySQL, Kafka** need storage
2. Pods can restart, reschedule, die anytime
3. Data must:
    - survive restarts
    - survive rescheduling
    - sometimes survive pod deletion

Without proper storage:

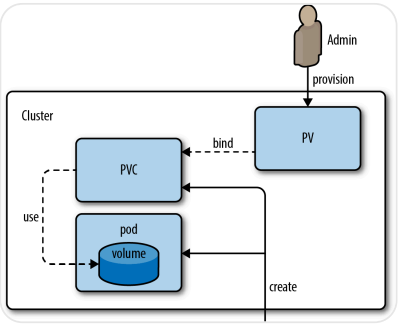
> Kubernetes is great at killing your data 😬

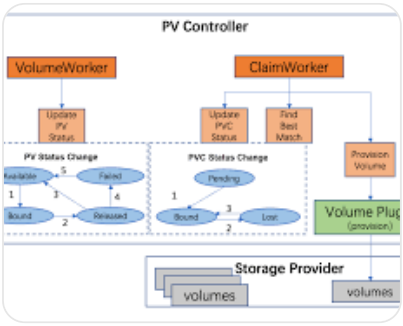
🛠️ How Kubernetes Solves This

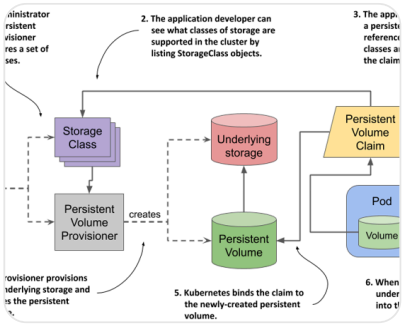
Kubernetes introduces **Volumes**, then **Persistent Volumes**.

Storage layers (important):

```
Container → Pod → Volume → PV → Physical Storage
```







Step 1 – emptyDir (Pod-lifecycle storage)

Your example 👇

`redis.yaml`

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: redis-pod
spec:
  volumes:
    - name: redis-storage
      emptyDir: {}
  containers:
  - image: redis
    name: redis-pod-container
```

```yaml
    volumeMounts:
    - name: redis-storage
      mountPath: /data/redis
```

What `emptyDir: {}` means

- Storage created **when POD is created**
- Storage deleted **when POD is deleted**
- Storage **survives container restart**

📌 **Key rule (very important):**

> `emptyDir` lives for **POD lifecycle**,
> NOT container lifecycle.

Q: Container restarts → data?

✅ **Data is retained**

Because:

- Volume is attached to **pod**
- Not to container

Q: Pod deleted → data?

× **Data is gone**

Because:

```kotlin
Pod deleted → emptyDir deleted → data deleted
```

Q: Liveness / readiness probe failure?

- Probe failure → **container restarts**
- Pod stays alive
- Volume stays alive

✅ Data safe

🚨 emptyDir use cases (REAL)

- Cache
- Temporary files
- Scratch space
- Build artifacts
- Sidecar communication

× Not for databases

× Not for production state

Step 2 – Why Persistent Storage is needed

If you delete the pod:

```bash
kubectl delete pod redis-pod
```

💥 **All data lost**

That's unacceptable for:

- Databases
- Message queues
- Stateful apps

So Kubernetes separates responsibilities 👇

Step 3 – Storage Responsibility Split (VERY IMPORTANT)

Role	Responsibility
Storage Admin	Creates **PV**
App/User	Creates **PVC**

Role	Responsibility
Kubernetes	Binds PV ↔ PVC

This separation is **intentional**.

Step 4 – Persistent Volume (PV)

PV = **actual storage**

Created by:

- Storage admin
- Or cloud provisioner

Example: `pv.yaml`

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-demo
spec:
  storageClassName: ""
  capacity:
    storage: 512Mi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/data/config"
```

What this means

Field	Meaning
`capacity`	Total storage available
`accessModes`	How it can be mounted
`hostPath`	Physical location (node disk)

Field	Meaning
`storageClassName: ""`	Static provisioning

⚠️ `hostPath` = **single-node only**
⚠️ Use only for learning / labs

Step 5 – Persistent Volume Claim (PVC)

PVC = **request for storage**

Created by:

- Developer / App team

Example: `pvc.yaml`

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-demo
spec:
  storageClassName: ""
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 256Mi
```

What PVC says

> "I need **256Mi**,
> with **ReadWriteMany**,
> from any matching PV."

Step 6 – PV ↔ PVC Binding Logic (CRITICAL)

Kubernetes matches PV & PVC based on:

1. **storageClassName**
2. **capacity**
3. **accessModes**

If all match:

```nginx
PVC → Bound
PV  → Bound
```

Example (your case)

- PV = 512Mi
- PVC = 256Mi

✅ Binding happens
× Remaining **256Mi is NOT reusable**

> One PV = one PVC
> No sharing leftover space

(This is a very common misunderstanding)

PVC bigger than PV?

```ini
PV  = 82Gi
PVC = 90Gi
```

⏳ **PVC stays Pending**

AccessMode mismatch?

```ini
```

```
PV  = ReadWriteOnce
PVC = ReadWriteMany
```

⏳ **PVC stays Pending**

Step 7 – Attach PVC to Pod

Example: `pod.yaml`

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  volumes:
    - name: pv-demo-storage
      persistentVolumeClaim:
        claimName: pvc-demo
  containers:
    - name: app-container
      image: nginx
      volumeMounts:
        - name: pv-demo-storage
          mountPath: "/var/app/config"
```

What happens now

1. PVC already bound to PV
2. PV attached to pod
3. Pod mounts storage at:

```arduino
/var/app/config
```

🔥 Now the BIG difference

Action	emptyDir	PV/PVC
Container restart	Data stays	Data stays
Pod restart	Data stays	Data stays
Pod delete	× gone	✅ stays
Pod recreate	× gone	✅ reattach

This is the **entire point** of PV/PVC.

Step 8 – Reclaim Policy (VERY IMPORTANT)

Each PV has a **reclaimPolicy**:

Policy	Meaning
Retain	Keep data (manual cleanup)
Delete	Delete storage
Recycle	(deprecated)

If PVC is deleted:

- **Retain** → data stays
- **Delete** → data wiped

⚠️ Default depends on storage class

Step 9 – Static vs Dynamic Provisioning

Static Provisioning

- Admin manually creates PV
- PVC binds to it

Your example = **static**

Dynamic Provisioning

- PVC created
- StorageClass creates PV automatically

Used in:

- AWS EBS
- GCP PD
- Azure Disk

Production standard ✅

Step 10 – NodeName trap (VERY IMPORTANT)

If you specify:

```yaml
spec:
  nodeName: worker-1
```

Then:

- Scheduler is bypassed
- Taints are ignored
- Pod is forced to node

⚠️ Dangerous with storage
⚠️ Can break HA & rescheduling

Step 11 – Common Mistakes (Seen DAILY)

× Using emptyDir for DB
× Expecting leftover PV space to be reused
× AccessMode mismatch confusion
× Using hostPath in production
× Forgetting reclaimPolicy
× Thinking PVC controls storage (PV does)

🧠 Final Mental Model (Lock this)

```kotlin
emptyDir → pod-lifecycle storage
PV/PVC   → independent data lifecycle
```

Or even simpler:

> **Pods are temporary.**
> **Volumes can be temporary.**
> **PersistentVolumes are not.**

If you want next 🔥

- StorageClass deep dive
- StatefulSet + PVC
- VolumeBindingMode
- RWX vs RWO in cloud
- CKA storage traps

Say the word 😏